

MotionPainter: Penumbra & Visibility Integration — Companion Document

Prepared as a companion/coda to the MotionPainter Canonical Guide (v2 and updates). This document explains the visiblefraction (penumbra/umbra) model, practical methods to compute it from camera frames, how to integrate the results into MotionPainter modules, and why mirrored ('enantiomeric') sets are useful. It is written to be pragmatic and implementation-ready.

Executive summary

We formalize an observable called the visiblefraction $f(P)$ for each image point (pixel) P . $f(P)$ in $[0,1]$ is the fraction (by area or radiance) of an extended source L that is visible via unobstructed straight segments to P . Using calibrated frames, motion probes, and (when available) depth, MotionPainter can compute coarse or high-resolution f maps and a companion directness field. These maps (artifacts) help PackingSDF, Minimizer, AmbiAdapter, and the CompositeRenderer make lighting-aware decisions: seed primitives on penumbral edges, bias optimizers to preserve direct cues, and guide the hybrid render policy to reuse frames or diversify temporal samples.

Key definitions & conceptual model

Source (L): an extended light patch or set in world space. Not a point emitter.

Occluders (O): geometry (or approximate depth) that blocks straight rays.

Visible subset $V(P)$: for pixel P , the subset of source points s in L such that segment $s-P$ is unobstructed.

Visible fraction $f(P)$: $\mu(V(P)) / \mu(L)$, where μ is an area/radiance measure on L . $f = 1 \Rightarrow$ fully lit; $0 < f < 1 \Rightarrow$ penumbra; $f = 0 \Rightarrow$ geometric umbra.

Direct irradiance $D(P)$: irradiance at P due to unobstructed source parts. Indirect irradiance $S(P)$: remainder from scattering, interreflection, diffraction or other transport.

Why this helps MotionPainter

1. Stability: seeding disks and SDF narrow bands around penumbral contours reduces flicker and preserves perceptual soft edges that artists care about. 2. Render policy: direct-dominant pixels are stable and can reuse frames; indirect-dominant pixels benefit from temporal mixing. 3. Telemetry: f maps are compact diagnostics for lighting changes and are useful for pose bucketing and LUT selection. 4. AmbiAdapter / Minimizer: bias optimization to honour direct illumination when generating slats and LUTs.

Computing I (the penumbra set) from camera frames — three practical routes

We offer three operational modes depending on the data available: (A) depth available, (B) camera + controlled motion, (C) pure image heuristics. Choose one or combine them for robustness.

A — Depth available: MonteCarlo source sampling + occlusion tests (recommended)

Inputs: calibrated frame, depth map (per-pixel), intrinsics, approximate source patch L (image coordinates or 3D).

Algorithm (high level):

- Sample N points s_i on the source patch L ($N=64\dots 512$ importance sampled).
- For each pixel P , unproject to world point P_w using depth + intrinsics.
- Raycast segment $s_i \leftrightarrow P_w$ against scene depth/mesh; count visible samples.
- $f(P)$ approx $\text{visible_count} / N$. Store f map (Float16) and directness = estimated direct irradiance from visible samples.

Notes: accelerate occlusion checks with BVH, voxelization, or depth buffer linear tests. Compute coarse low-res maps for runtime; push high-res compute to a worker.

B — Camera or source motion probing (no explicit depth)

Use a small, controlled camera/lamp motion (hand-jiggle, pan, or lamp shift) recorded in short bursts.

Procedure (practical):

- Track the lamp centroid in image coords (lightTrack artifact).
- For each pixel, compute temporal correlation between known lamp motion vector and intensity changes (ΔI). High correlation $\Rightarrow$ likely direct.
- Detect soft vs sharp shadow edges with edge profiling (sample orthogonal intensity profile, fit logistic). Wide transition widths indicate penumbra.
- Combine motion correlation and edge width into a directness proxy map.

This is cheap and runs on PreprocessorWorker or MotionWorker. It produces coarse but useful f proxies.

C — Pure image/statistics (fallback)

When neither depth nor motion probes are available, use spatial cues:

- Shadow edge width and curvature of iso-luminance contours (penumbral bulges have smooth curvature).
- Intrinsic image decomposition (albedo vs shading) to isolate illumination.
- Classify pixels into UMBRA / PENUMBRA / FULLY_LIT using gradient magnitudes and local variance heuristics.

This route is heuristic and should be combined with motion or depth if possible.

Artifact schemas to persist (recommended)

All artifacts follow existing MotionPainter storage patterns (versioned keys, small payloads, and ImageBitmap/transferables where heavy). Suggested keys:

mp:artifact:calibratedFrame:{frameKey}:v1 — linearized image (ImageBitmap/Blob), metadata.

mp:artifact:lightTrack:{frameKey}:v1 — [{id, imageXY, conf, radius, centroid}].

mp:artifact:directness:{frameKey}:v1 — {directnessMap: Float16Array, fMap: Float16Array, meta:{N_samples}}.

mp:artifact:fresnel:{frameKey}:v1 — existing Fresnel annular densityMap used by PackingSDF (seed).

mp:artifact:packingSdf:{frameKey}:v1 — signed_sdf, narrowMask, disks[] (reuse existing guide schema).

Message contract example (Preprocessor -> MotionWorker)

```
{ "type": "analysis:lightAndDirectness", "frameKey": "mp:frame:2025-10-05T12:34:56Z",
  "calibratedFrameKey": "mp:artifact:calibratedFrame:...:v1",
  "lightTrackKey": "mp:artifact:lightTrack:...:v1",
  "directnessKey": "mp:artifact:directness:...:v1", "telemetry": { "samples": 128, "timeMs": 42 } }
```

How PackingSDF / Minimizer / AmbiAdapter should use directness maps

- PackingSDF: use fMap as densityMap to seed disk packing; prefer disk centers on penumbral curvature maxima; use narrowMask to focus SDF compute.
- Minimizer: add a soft penalty in the objective for primitives that cut high directness regions; treat penumbral edges as soft constraints to be matched by slats or gradients.
- AmbiAdapter: use enantiomeric priors (see below) and keypoint heatmaps together with disks to optimize slat alignment; preserve direct lighting cues in LUT generation to maintain perceptual consistency.

Direct vs Indirect split — estimation and use

If depth is available, compute direct irradiance $D(P)$ by summing contributions from visible source samples with geometry terms (Lambert, $1/r^2$). Measured irradiance $I_m(P)$ (from calibrated frame) minus $D(P)$ gives $S(P)$ (indirect).

If depth is not available, use lamp modulation (toggle intensity) and measure responsive fraction $R = \Delta I_m / \Delta I_{lamp}$; estimate $D \approx R * I_m$.

Use the ratio $D/(D+S)$ as directDominance. Thresholds: >0.9 direct, $0.1-0.9$ penumbra/mix, <0.1 indirect/umbra.

Enantiomeric sets — definition and usefulness

Definition: let R be a reflection isometry (plane reflection). For any set A (L, O, or image sample set I), define its enantiomer $A' = R(A)$. At image level: reflect 3D points then reproject; image flip alone is an approximation.

Why build enantiomeric sets? Practical motivations:

- Symmetry priors: many scenes and targets (faces, objects) are approximately symmetric. Mirrored artifacts help AmbiAdapter test candidate slat solutions under symmetric assumptions and speed up optimization by halving search space.
- Robustness: comparing I and its enantiomer E helps detect asymmetric lighting or chiral features (useful heuristics for deciding whether an ambigram should prefer symmetric slat placement).
- Data augmentation: mirrored fMaps and disks provide additional training examples for learning-based components and stabilize pose bucket LUT interpolation.
- Debug & QA: mirror contrast between I and E highlights errors in depth/occluder reconstruction (mismatch indicates pose or depth bias).

Telemetry & diagnostics to collect per frame

Collect compact JSON telemetry alongside artifacts:

- mean/std of fMap, penumbra area fraction, histogram bins of f values.
- lightTrack: number of lights, confidences, centroid motion.

- compute cost: samples×pixels, workerTimeMs, queueDepth.
 - PackingSDF stability: disk count, mean radius, disk jitter score.
- Persist telemetry to Vault as short JSON for offline analysis and tuning.

Rollout plan (prioritized)

Quick wins (days):

- Add calibratedFrame artifact and simple image-space penumbra heuristics (edge width).
- Implement lightTrack (fast centroid + radius) and emit as artifact.
- Add telemetry counters and dev overlay for penumbra visualization.

Mid term (weeks):

- Motion probing directness in MotionWorker; produce coarse directnessMap; hook into PackingSDF seeding.
- Integrate directnessMap into hybrid render policy (reuse/directness rules).

Long term (months):

- Full MonteCarlo visibility worker using depth/mesh; high-res fMaps and pose-bucketed LUTs.
- Automatic tuning of hybrid policy using telemetry and gradient-free optimizers.

Practical parameters & heuristics (recommended defaults)

Parameter	Recommended start
Source samples N	64–128 (coarse), 256+ (high quality)
Directness quantization	Float16 map, 8/16 bit quantized cache
penumbra width detection	profile window 9–25 px, fit logistic
SDF raster size	1024×1024 (high), 512×512 (fast)
Pose bucket granularity	~1° / 1cm
Disk radii (world units)	rMin=0.005, rMax=0.1 (tunable)

Example MonteCarlo pseudocode (depth case)

```
for each pixel (x,y): Pw = unproject(x,y, depthMap, intrinsics) visible = 0 directI = 0
for s in sourceSamples: if not intersectsOccluder(Pw, s, sceneMesh): visible += 1 directI +=
radiance(s) * lambertTerm(Pw, s) / (distance^2) f = visible / N store fMap[x,y] = f store
directnessMap[x,y] = directI
```

Message contracts & storage notes (short)

Follow existing conventions in the Canonical Guide (versioned keys and feature flags). Example artifact keys and minimal message shown earlier. Transfer large buffers (fMap) as Float16Array via transferable ArrayBuffers; send only compact thumbnails to UI.

References & traceability

This companion builds on and integrates into the Motion■Painter Canonical Guide and its updates. See the canonical documents in the Vault for full module specs and API examples:

- MotionPainter_Canonical_Guide.pdf [filecite]
- MotionPainter_Canonical_Guide_v2.pdf [filecite]
- MotionPainter_Canonical_Guide_Updated.pdf [filecite]

Prepared for the Motion■Painter team. Implementation■ready; worker stubs and code can be supplied on request. If you want the Monte■Carlo worker implementation or a motion■probing JS stub next, I can include it in an appendix or as a separate artifact.